

Bloque 2 — Principales sistemas de almacenamiento y formatos

Módulo: Extracción, Transformación y Carga de Datos desde Fuentes Múltiples (5104) **Unidad**

Didáctica: UD1 — Tipología y características de las fuentes de datos **Duración estimada:** ~6 horas

Curso académico: 2026-2027

Contenidos de este bloque

Epígrafe	Contenido
2.1	Bases de datos relacionales (SQL): modelo relacional y uso en ML
2.2	Bases de datos NoSQL: documentos, clave-valor, columnar y grafos
2.3	Formatos de intercambio: CSV, JSON y XML
2.4	Formatos orientados a ML: Parquet, Apache Arrow y HDF5
2.5	Sistemas IoT y SCADA: series temporales y ML industrial

Criterios de evaluación asociados

(CE d) Se han descrito los sistemas gestores de datos SQL y no SQL, diferenciando sus estructuras, modelos y aplicaciones actuales. (CE e) Se han analizado los formatos de texto estructurados para el intercambio de datos, tales como ficheros planos, XML y JSON. (CE f) Se han determinado el formato y la utilidad de los datos procedentes de sistemas SCADA aplicados en IoT.

Librerías cargadas correctamente.

2.1 Bases de datos relacionales (SQL)

El **modelo relacional** organiza los datos en **tablas** (relaciones) formadas por filas (tuplas) y columnas (atributos). Su fortaleza es la **integridad referencial**: las claves foráneas garantizan que los datos son consistentes entre tablas.

En proyectos de ML, las BD relacionales son la fuente más común de datos tabulares estructurados listos para entrenar. El flujo habitual es:

```
BD relacional --> pd.read_sql_query(sql, conn) --> DataFrame -->
sklearn / XGBoost / ...
```

Principales sistemas y cuándo usarlos en ML

SGBD	Tipo de despliegue	Caso de uso en ML	Característica clave
SQLite	Fichero local, sin servidor	Prototipado, tests, cuadernos	Cero configuración
PostgreSQL	Servidor gestionado o nube	Producción corporativa, feature stores	Extensiones (pgvector, PostGIS)
MySQL/MariaDB	Servidor web	Datos de aplicaciones web y e-commerce	Ecosistema muy amplio
DuckDB	Embebido (como SQLite)	SQL analítico sobre Parquet/CSV, ETL	Velocidad OLAP en local

DuckDB es el gran protagonista emergente: permite hacer SQL analítico directamente sobre ficheros Parquet o CSV sin cargar todo en memoria. Ideal para exploración inicial de datasets grandes en un cuaderno.

Principales SGBD relacionales y su rol en ML

SQLite <i>Fichero local, sin servidor</i> - Prototipado rapido - Cuadernos jupyter - Tests unitarios - Apps moviles Cero configuracion	PostgreSQL <i>Servidor avanzado, extensible</i> - Feature stores (pgvector) - Produccion corporativa - Datos geograficos (PostGIS) - ML en la BD Potencia + extensiones	MySQL / MariaDB <i>Alta disponibilidad web</i> - Datos de webs y apps - Logs de usuarios - E-commerce y catalogs - CMS y CRMs Ecosistema enorme	DuckDB <i>OLAP embebido, analítico</i> - SQL sobre Parquet / CSV - ETL en cuadernos - Benchmark de datasets - Sin servidor Velocidad analítica local
---	--	--	---

```
# — Ejemplo: diseno de base de datos para un proyecto ML de e-commerce —
#
# Problema: predecir si un cliente va a abandonar la plataforma (churn).
# Los datos estan en varias tablas relacionales. Necesitamos un JOIN para
# construir la tabla de features que alimenta el modelo.
```

```
conn = sqlite3.connect(':memory:')

conn.executescript("""
-- Tabla de clientes
CREATE TABLE clientes (
    id            INTEGER PRIMARY KEY,
    nombre        TEXT,
    pais          TEXT,
    fecha_alta    TEXT      -- ISO-8601: YYYY-MM-DD
);

-- Tabla de pedidos
CREATE TABLE pedidos (
    id            INTEGER PRIMARY KEY,
    cliente_id    INTEGER REFERENCES clientes(id),
    fecha         TEXT,
    importe       REAL,
```

```

    completado INTEGER -- 0=cancelado, 1=completado
);

-- Tabla de incidencias (soporte)
CREATE TABLE incidencias (
    id INTEGER PRIMARY KEY,
    cliente_id INTEGER REFERENCES clientes(id),
    tipo TEXT, -- 'retraso', 'devolucion', 'otro'
    resuelta INTEGER
);

-- Datos de ejemplo
INSERT INTO clientes VALUES
(1, 'Ana', 'ES', '2023-01-15'),
(2, 'Blas', 'ES', '2022-06-01'),
(3, 'Clara', 'FR', '2023-09-20'),
(4, 'David', 'ES', '2021-03-10'),
(5, 'Eva', 'DE', '2023-11-05');

INSERT INTO pedidos VALUES
(1, 1, '2024-01-10', 89.50, 1),
(2, 1, '2024-02-05', 120.00, 1),
(3, 1, '2024-03-01', 45.00, 0),
(4, 2, '2024-01-20', 210.00, 1),
(5, 2, '2024-04-12', 30.00, 1),
(6, 3, '2024-02-28', 55.00, 1),
(7, 4, '2023-12-01', 450.00, 1),
(8, 4, '2024-01-15', 200.00, 1),
(9, 4, '2024-02-20', 180.00, 0),
(10, 5, '2024-01-08', 75.00, 1);

INSERT INTO incidencias VALUES
(1, 1, 'retraso', 1),
(2, 1, 'devolucion', 0),
(3, 2, 'otro', 1),
(4, 4, 'retraso', 1),
(5, 4, 'devolucion', 1);
""")

# Consulta SQL que construye la tabla de features para churn prediction
# Usamos agregaciones y LEFT JOIN para combinar las tres tablas
query = """
SELECT
    c.id AS cliente_id,
    c.pais,
    CAST(
        julianday('2024-05-01') - julianday(c.fecha_alta)
        AS INTEGER
    ) AS dias_cliente,
    COUNT(DISTINCT p.id) AS n_pedidos,
    COALESCE(SUM(p.importe), 0) AS gasto_total,
    COALESCE(AVG(p.importe), 0) AS ticket_medio,
    COALESCE(SUM(CASE WHEN p.completado=0 THEN 1 ELSE 0 END), 0) AS pedidos_cancelados,
    COALESCE(COUNT(DISTINCT i.id), 0) AS n_incidentes,
    COALESCE(SUM(CASE WHEN i.resuelta=0 THEN 1 ELSE 0 END), 0) AS incid_abiertas
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
LEFT JOIN incidencias i ON i.cliente_id = c.id

```

```
GROUP BY c.id, c.pais, c.fecha_alta
ORDER BY c.id
"""
```

```
df_features = pd.read_sql_query(query, conn)
print("Tabla de features para churn prediction (extraida via SQL):")
print(df_features.to_string(index=False))
print()
print("Esta tabla ya esta lista para sklearn, XGBoost o cualquier modelo tabular.")
print("Cada fila = un cliente. Columnas = features numericas + categoricas.")
conn.close()
```

Tabla de features para churn prediction (extraida via SQL):

cliente_id	pais	dias_cliente	n_pedidos	gasto_total	ticket_medio
pedidos_cancelados	n_incidencias	incid_abiertas			
1	ES	472	3	509.0	
84.833333		2	2		3
2	ES	700	2	240.0	
120.000000		0	1		0
3	FR	224	1	55.0	
55.000000		0	0		0
4	ES	1148	3	1660.0	
276.666667		2	2		0
5	DE	178	1	75.0	
75.000000		0	0		0

Esta tabla ya esta lista para sklearn, XGBoost o cualquier modelo tabular.
Cada fila = un cliente. Columnas = features numericas + categoricas.

— DuckDB: SQL analitico sobre ficheros CSV/Parquet sin servidor —

```
try:
    import duckdb

    # Creamos un DataFrame grande de ejemplo y lo guardamos como CSV temporal
    rng = np.random.default_rng(42)
    n = 100_000
    df_grande = pd.DataFrame({
        'producto_id': rng.integers(1, 500, n),
        'categoria':   rng.choice(['A', 'B', 'C', 'D'], n),
        'precio':      rng.uniform(5, 500, n).round(2),
        'cantidad':    rng.integers(1, 20, n),
        'descuento':   rng.uniform(0, 0.3, n).round(3),
        'devuelto':    rng.integers(0, 2, n),
    })
    df_grande.to_csv('/tmp/ventas.csv', index=False)

    # DuckDB puede hacer SQL directamente sobre el CSV (sin cargarlo todo en pandas)
    con = duckdb.connect()

    resultado = con.execute("""
        SELECT
            categoria,
            COUNT(*)                                AS n_ventas,
            ROUND(AVG(precio), 2)                   AS precio_medio,
            ROUND(SUM(precio * cantidad), 0)         AS facturacion,
            ROUND(AVG(descuento) * 100, 1)           AS descuento_medio_pct,
            ROUND(AVG(devuelto) * 100, 1)            AS tasa_devolucion_pct
        FROM read_csv_auto('/tmp/ventas.csv')
        GROUP BY categoria
        ORDER BY facturacion DESC
    """)
```

```

""").df()

print("DuckDB: SQL analitico directo sobre CSV (100.000 filas):")
print(resultado.to_string(index=False))
print()
print("-> DuckDB lee el CSV con proyeccion de columnas y filtrado pushdown.")
print("    Solo carga en memoria las columnas necesarias para la consulta.")

except ImportError:
    print("DuckDB no instalado. Para instalarlo: pip install duckdb")
    print()
    print("Equivalente con pandas (carga todo en memoria):")
    rng = np.random.default_rng(42)
    n = 100_000
    df_grande = pd.DataFrame({
        'producto_id': rng.integers(1, 500, n),
        'categoria':   rng.choice(['A', 'B', 'C', 'D'], n),
        'precio':      rng.uniform(5, 500, n).round(2),
        'cantidad':    rng.integers(1, 20, n),
        'descuento':   rng.uniform(0, 0.3, n).round(3),
        'devuelto':    rng.integers(0, 2, n),
    })
    resultado = (df_grande.groupby('categoria')
        .agg(n_ventas=('precio', 'count'),
            precio_medio=('precio', 'mean'),
            facturacion=('precio', lambda x: (x * df_grande.loc[x.index,
                descuento_medio_pct=('descuento', lambda x: x.mean() * 100),
                tasa_devolucion_pct=('devuelto', lambda x: x.mean() * 100))
            .round(2).reset_index())
    print(resultado.to_string(index=False))

```

DuckDB no instalado. Para instalarlo: pip install duckdb

Equivalente con pandas (carga todo en memoria):

	categoria	n_ventas	precio_medio	facturacion	descuento_medio_pct	tasa_devolucion_pct
50.13	A	24786	252.21	62816769.03	14.92	
50.29	B	25058	252.44	63362522.96	15.02	
49.83	C	25169	251.24	62995213.43	14.88	
49.93	D	24987	252.80	63296464.36	15.06	

2.2 Bases de datos NoSQL

El término **NoSQL** agrupa sistemas de almacenamiento que no siguen el modelo relacional estricto.

Nacieron para resolver tres limitaciones del modelo relacional en la era del Big Data:

1. **Escalabilidad horizontal:** añadir más nodos en lugar de un servidor más potente.
2. **Esquema flexible:** no necesitar definir la estructura antes de insertar.
3. **Modelos de datos especializados:** grafos, series temporales, vectores...

Cuatro familias principales

Familia	Estructura	Ejemplos	Uso en ML
Documental	Documentos JSON/BSON anidados	MongoDB, Firestore, CouchDB	Perfiles de usuario, catálogos, logs
Clave-Valor	Par {clave: valor} con lookup O(1)	Redis, DynamoDB, Memcached	Feature stores, cache de predicciones
Columnar	Familias de columnas, escritura masiva	Cassandra, HBase, BigTable	Series temporales, telemetría IoT
Grafos	Nodos y aristas con propiedades	Neo4j, Amazon Neptune	Detección de fraude, knowledge graphs, RAG

En el contexto de LLMs (2024+) ha emergido una quinta familia: las **bases de datos vectoriales** (Pinecone, Chroma, pgvector), especializadas en búsqueda por similitud semántica. Las veremos en UD4.

CAP Theorem: el trilema del almacenamiento distribuido

Los sistemas NoSQL son diseñados priorizando dos de estas tres propiedades:

- Consistencia — todos leen el mismo dato
- Availability — siempre hay respuesta
- Partition tolerance — funciona con fallos de red



```
# — Ejemplo: base de datos documental (simulacion tipo MongoDB) —
#
# MongoDB almacena "documentos" JSON/BSON. Cada documento puede tener
# campos distintos. Aqui simulamos con dicts de Python la misma logica.

print("=== Base de datos documental: catalogo de modelos ML ===")
print()

# Coleccion de documentos (equivale a una "tabla" en MongoDB)
modelos_ml = [
    {
```

```

        "_id": "model_001",
        "nombre": "XGBoost Churn v2.1",
        "tipo": "clasificacion",
        "metricas": {"roc_auc": 0.91, "f1": 0.84, "precision": 0.87},
        "features": ["dias_cliente", "n_pedidos", "gasto_total", "n_incidentes"],
        "hiperparametros": {"n_estimators": 300, "max_depth": 6, "learning_rate": 0.01},
        "fecha_entrenamiento": "2024-03-15",
        "activo": True
    },
    {
        "_id": "model_002",
        "nombre": "RandomForest Precios v1.0",
        "tipo": "regresion",
        "metricas": {"rmse": 12.4, "mae": 8.9, "r2": 0.88},
        "features": ["categoria", "peso_kg", "temporada", "competencia_precio"],
        "hiperparametros": {"n_estimators": 200, "max_depth": None},
        "fecha_entrenamiento": "2024-01-20",
        "activo": False,
        "nota": "Superado por LightGBM v2" # campo opcional: solo en este doc
    },
    {
        "_id": "model_003",
        "nombre": "LightGBM Precios v2.0",
        "tipo": "regresion",
        "metricas": {"rmse": 9.1, "mae": 6.3, "r2": 0.93},
        "features": ["categoria", "peso_kg", "temporada", "competencia_precio", "stadium"],
        "hiperparametros": {"num_leaves": 63, "learning_rate": 0.03, "n_estimators": 100},
        "fecha_entrenamiento": "2024-04-10",
        "activo": True
    }
]

# Consulta 1: filtrar por tipo (equivale a collection.find({tipo: "clasificacion"}))
print("Consulta: modelos activos de clasificacion")
print("-" * 50)
activos_clasificacion = [m for m in modelos_ml
                        if m["activo"] and m["tipo"] == "clasificacion"]
for m in activos_clasificacion:
    print(f" {m['nombre']} | ROC-AUC: {m['metricas']['roc_auc']}")

# Consulta 2: extraer campo anidado (metricas) para comparar
print()
print("Comparativa de modelos activos por metrica principal:")
print("-" * 50)
for m in [m for m in modelos_ml if m["activo"]]:
    metrica_clave = "roc_auc" if m["tipo"] == "clasificacion" else "r2"
    print(f" [{m['tipo'][:5]}] {m['nombre'][:30]} {metrica_clave}: {m['metricas'][metrica_clave]}")

# Ventaja de los documentos: podemos convertir a DataFrame con json_normalize
df_modelos = pd.json_normalize(modelos_ml, sep='_')
print()
print("Columnas tras json_normalize (aplanado automatico de campos anidados):")
print([c for c in df_modelos.columns if 'metricas' in c or c in ['_id', 'nombre', 'tipo', 'fecha_entrenamiento', 'activo']])

```

=== Base de datos documental: catalogo de modelos ML ===

Consulta: modelos activos de clasificacion

XGBoost Churn v2.1 | ROC-AUC: 0.91

Comparativa de modelos activos por metrica principal:

[clasi] XGBoost Churn v2.1 roc_auc: 0.91
[regre] LightGBM Precios v2.0 r2: 0.93

Columnas tras json_normalize (aplanado automatico de campos anidados):
['_id', 'nombre', 'activo', 'metricas_roc_auc', 'metricas_f1',
'metricas_precision', 'metricas_rmse', 'metricas_mae', 'metricas_r2']

```
# — Ejemplo: almacen clave-valor (simulacion tipo Redis) —————
#
# Redis almacena pares {clave: valor}. Es extremadamente rapido (O(1)).
# En ML se usa como feature store online: el modelo pide las features
# de un usuario en tiempo real durante la inferencia.

print("=== Feature Store online (patron clave-valor tipo Redis) ===")
print()

class FeatureStoreSimulado:
    """Simulacion de un feature store tipo Redis para inferencia en tiempo real."""

    def __init__(self):
        self._store = {}

    def set_features(self, entidad_id: str, features: dict, ttl_s: int = None):
        """Guarda las features de una entidad. Clave = 'features:{entidad_id}' """
        clave = f"features:{entidad_id}"
        self._store[clave] = features
        print(f" SET {clave} -> {len(features)} features guardadas")

    def get_features(self, entidad_id: str) -> dict:
        """Recupera las features para inferencia. Lookup O(1). """
        clave = f"features:{entidad_id}"
        return self._store.get(clave, {})

    def bulk_set(self, entidades: dict):
        """Escritura masiva (pipeline en Redis real). """
        for eid, feats in entidades.items():
            self.set_features(eid, feats)

    def __len__(self):
        return len(self._store)

fs = FeatureStoreSimulado()

# Escritura: el pipeline de entrenamiento calcula features y las escribe
print("1. Escritura de features (pipeline batch, cada 24h):")
fs.bulk_set({
    "cliente_1001": {"dias_cliente": 320, "n_pedidos": 12, "gasto_total": 890.5,
                     "ticket_medio": 74.2, "n_incidentes": 2, "pais_ES": 1},
    "cliente_1002": {"dias_cliente": 45, "n_pedidos": 3, "gasto_total": 120.0,
                     "ticket_medio": 40.0, "n_incidentes": 0, "pais_ES": 0},
    "cliente_1003": {"dias_cliente": 892, "n_pedidos": 67, "gasto_total": 4500.0,
```

```

        "ticket_medio": 67.2, "n_incidentes": 5, "pais_ES": 1},
    })

print()
print(f"    Feature store contiene {len(fs)} entidades.")

# Lectura: el modelo pide features cuando llega una solicitud de prediccion
print()
print("2. Lectura de features (API de inferencia, tiempo real):")
for cid in ["cliente_1001", "cliente_1002"]:
    feats = fs.get_features(cid)
    print(f"    GET {cid}: {feats}")

print()
print("-> Ventaja clave: el modelo no recalcula features en cada inferencia.")
print("    Las features se precaculan (batch) y se leen en O(1) desde Redis.")
print("    Esto evita el 'training-serving skew': mismas features en train y prod.")

```

=== Feature Store online (patron clave-valor tipo Redis) ===

1. Escritura de features (pipeline batch, cada 24h):

```

SET features:cliente_1001 -> 6 features guardadas
SET features:cliente_1002 -> 6 features guardadas
SET features:cliente_1003 -> 6 features guardadas

```

Feature store contiene 3 entidades.

2. Lectura de features (API de inferencia, tiempo real):

```

GET cliente_1001: {'dias_cliente': 320, 'n_pedidos': 12, 'gasto_total': 890.5,
'ticket_medio': 74.2, 'n_incidentes': 2, 'pais_ES': 1}
GET cliente_1002: {'dias_cliente': 45, 'n_pedidos': 3, 'gasto_total': 120.0,
'ticket_medio': 40.0, 'n_incidentes': 0, 'pais_ES': 0}

```

-> Ventaja clave: el modelo no recalcula features en cada inferencia.

Las features se precaculan (batch) y se leen en O(1) desde Redis.

Esto evita el 'training-serving skew': mismas features en train y prod.

— Ejemplo: grafo de conocimiento con NetworkX (patron tipo Neo4j) —

```

try:
    import networkx as nx

    print("=== Grafo de transacciones para deteccion de fraude ===")
    print()

    # En fraude financiero, las cuentas y transacciones forman un GRAFO.
    # Neo4j permite consultas como "encuentra ciclos de transferencias
    # entre cuentas en menos de 24 horas" que son imposibles en SQL.

    G = nx.DiGraph()

    # Nodos: cuentas bancarias
    cuentas = {
        'A001': {'tipo': 'personal', 'pais': 'ES', 'score_riesgo': 0.1},
        'A002': {'tipo': 'empresa', 'pais': 'ES', 'score_riesgo': 0.3},
        'A003': {'tipo': 'personal', 'pais': 'RU', 'score_riesgo': 0.8},
        'A004': {'tipo': 'empresa', 'pais': 'ES', 'score_riesgo': 0.2},
        'A005': {'tipo': 'personal', 'pais': 'RU', 'score_riesgo': 0.9},
    }

    for cuenta_id, attrs in cuentas.items():
        G.add_node(cuenta_id, **attrs)

```

```

# Aristas: transferencias (origen -> destino, con importe)
transferencias = [
    ('A001', 'A002', 1500),
    ('A002', 'A003', 1400), # <- sospechoso: A002 reenvía casi todo
    ('A003', 'A005', 1380), # <- sospechoso: cadena hacia RU
    ('A001', 'A004', 200),
    ('A004', 'A001', 195), # <- ciclo pequeño entre A001 y A004
    ('A003', 'A001', 50), # <- cierra el ciclo principal (fraude en anillo)
]

for origen, destino, importe in transferencias:
    G.add_edge(origen, destino, importe=importe)

# Features de grafo para ML
print("Features de grafo extraídas para cada cuenta:")
print(f"{'Cuenta':<8} {'In-degree':>10} {'Out-degree':>11} {'PageRank':>10} {'F"}
print("-" * 55)

pagerank = nx.pagerank(G, alpha=0.85)
for cuenta in sorted(G.nodes()):
    attrs = G.nodes[cuenta]
    pr = pagerank[cuenta]
    print(f"{'cuenta':<8} {'G.in_degree(cuenta):>10} {'G.out_degree(cuenta):>11} "
          f"{'pr':>10.4f} {'attrs['score_riesgo']:>8.1f}")

# Deteccion de ciclos (patron tipico de fraude en anillo)
ciclos = list(nx.simple_cycles(G))
print()
print(f"Ciclos detectados en el grafo: {len(ciclos)}")
for c in ciclos:
    print(f" Ciclo: {' -> '.join(c)} -> {c[0]}")
print()
print("-> En Neo4j: MATCH p=(a)-[:TRANSFIERE*2..5]->(a) RETURN p")
print(" Esta consulta de Cypher detecta ciclos en 1 linea.")
print(" En SQL requeriria CTEs recursivos muy complejos.")

except ImportError:
    print("NetworkX no instalado. Para instalarlo: pip install networkx")
    print()
    print("NetworkX permite crear grafos en Python y extraer features como:")
    print(" - Centralidad (PageRank, betweenness, closeness)")
    print(" - Deteccion de comunidades (Louvain, Girvan-Newman)")
    print(" - Deteccion de ciclos (patrones de fraude en anillo)")
    print(" - Caminos mas cortos entre nodos")

```

NetworkX no instalado. Para instalarlo: pip install networkx

NetworkX permite crear grafos en Python y extraer features como:

- Centralidad (PageRank, betweenness, closeness)
- Deteccion de comunidades (Louvain, Girvan-Newman)
- Deteccion de ciclos (patrones de fraude en anillo)
- Caminos mas cortos entre nodos

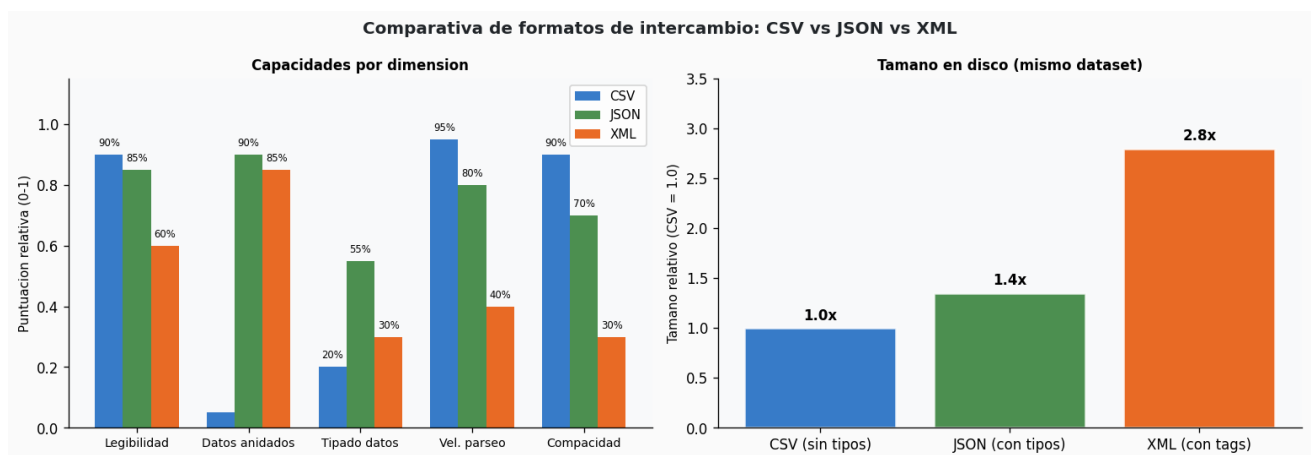
2.3 Formatos de intercambio de datos: CSV, JSON y XML

Los formatos de intercambio son la "lingua franca" entre sistemas: se usan para exportar datos de una BD, compartir datasets entre organizaciones o alimentar pipelines ML desde ficheros.

Comparativa de los tres formatos clásicos

Característica	CSV	JSON	XML
Legibilidad humana	Alta	Alta	Media
Soporte datos anidados	No	Sí	Sí
Tipado de datos	No (todo texto)	Básico (número, bool, null)	No (todo texto)
Tamaño relativo	Pequeño	Medio	Grande (tags verbosos)
Velocidad de parseo	Muy rápida	Rápida	Lenta
Estándar en ML	Sí (datasets tabulares)	Sí (APIs, configs)	Legacy (sectores específicos)
Pandas	<code>read_csv()</code>	<code>read_json()</code>	(via ElementTree)

⚠ **Trampas comunes en CSV:** separador incorrecto, encoding (UTF-8 vs Latin-1), cabecera en fila incorrecta, tipos mezclados en columnas, valores nulos representados de formas distintas (`NaN` , `N/A` , `-` , `NULL` ...). El 80 % del tiempo de preprocesado suele estar aquí.



```
# — CSV: lectura robusta con pandas —
#
# En la practica los CSV llegan con multitud de problemas. Aqui vemos
# como manejar los mas comunes.

import io

# CSV con problemas tipicos
csv_problemativo = """id;nombre;edad;salario;ciudad;fecha_alta
1;Ana Garcia;32;28500,50;Madrid;15/01/2023
2;Blas;N/A;31000.00;Barcelona;01-06-2022
3;Clara O'Brien;29;"27.800,00";Sevilla;20/09/2023
4;David;;42000;Valencia;
5;Eva Mora;41;-;Malaga;05/11/2023
"""

print("CSV original (con separador ';', encoding mixto, nulos variados):")
print(csv_problemativo)
```

```

# Lectura robusta
df = pd.read_csv(
    io.StringIO(csv_problematico),
    sep=';',
    na_values=['N/A', '-', ''],
    decimal=',',
    thousands='.',
    parse_dates=['fecha_alta'],
    dayfirst=True,
    dtype={'salario': str},
)

# Limpieza post-lectura del campo salario (mezcla de formatos)
def limpiar_salario(s):
    if pd.isna(s):
        return np.nan
    s = str(s).replace('.', '').replace(',', '.', '')
    try:
        return float(s)
    except ValueError:
        return np.nan

df['salario'] = df['salario'].apply(limpiar_salario)

print("DataFrame despues de lectura robusta:")
print(df.to_string())
print()
print(f"Tipos de datos: {dict(df.dtypes)}")
print(f"Valores nulos por columna:")
print(df.isnull().sum().to_string())
print()
print("-> Siempre explorar el CSV crudo antes de asumir que read_csv() lo")
print("  procesa correctamente. El parametro na_values es tu mejor amigo.")

```

CSV original (con separador ';', encoding mixto, nulos variados):

```
id;nombre;edad;salario;ciudad;fecha_alta
1;Ana Garcia;32;28500,50;Madrid;15/01/2023
2;Blas;N/A;31000.00;Barcelona;01-06-2022
3;Clara O'Brien;29;"27.800,00";Sevilla;20/09/2023
4;David;;42000;Valencia;
5;Eva Mora;41;-;Malaga;05/11/2023
```

DataFrame despues de lectura robusta:

	id	nombre	edad	salario	ciudad	fecha_alta
0	1	Ana Garcia	32.0	28500.5	Madrid	15/01/2023
1	2	Blas	NaN	31000.00	Barcelona	01-06-2022
2	3	Clara O'Brien	29.0	27800.0	Sevilla	20/09/2023
3	4	David	NaN	42000.0	Valencia	NaN
4	5	Eva Mora	41.0	NaN	Malaga	05/11/2023

Tipos de datos: {'id': dtype('int64'), 'nombre': dtype('O'), 'edad': dtype('float64'), 'salario': dtype('float64'), 'ciudad': dtype('O'), 'fecha_alta': dtype('O')}

Valores nulos por columna:

id	0
nombre	0
edad	2
salario	1
ciudad	0
fecha_alta	1

-> Siempre explorar el CSV crudo antes de asumir que read_csv() lo procesa correctamente. El parametro na_values es tu mejor amigo.

```
# — JSON: formatos avanzados (NDJSON, arrays de objetos) —————

# NDJSON (Newline-Delimited JSON): un objeto JSON por linea.
# Es el formato estandar para logs, eventos de usuario, streaming outputs.
# Ventaja: se puede procesar linea a linea sin cargar todo en memoria.

ndjson_logs = """\
{"ts": "2024-05-01T10:00:01Z", "user": "u001", "event": "login", "ok": true}
{"ts": "2024-05-01T10:00:45Z", "user": "u001", "event": "search", "query": "laptop"}
{"ts": "2024-05-01T10:01:10Z", "user": "u001", "event": "click", "item_id": "P-982"}
{"ts": "2024-05-01T10:02:00Z", "user": "u002", "event": "login", "ok": false, "motivo": "no autorizado"}
{"ts": "2024-05-01T10:03:15Z", "user": "u001", "event": "compra", "item_id": "P-982"}
"""

# Lectura de NDJSON linea a linea (eficiente para ficheros grandes)
registros = []
for linea in ndjson_logs.strip().split('\n'):
    registros.append(json.loads(linea))

df_logs = pd.DataFrame(registros)
df_logs['ts'] = pd.to_datetime(df_logs['ts'])
print("Logs en formato NDJSON convertidos a DataFrame:")
print(df_logs[['ts', 'user', 'event', 'ok', 'query', 'item_id']].to_string(index=False))

print()
print("Patron de sesion de u001 (secuencia de eventos en orden):")
sesion = df_logs[df_logs['user'] == 'u001'].sort_values('ts')
print(" -> ".join(sesion['event'].tolist()))
print()
```

```
# En pandas >= 1.0: pd.read_json con lines=True es equivalente
df_ndjson = pd.read_json(io.StringIO(ndjson_logs), lines=True)
print("Mismo resultado con pd.read_json(lines=True):")
print(df_ndjson[['ts', 'user', 'event']].to_string(index=False))
```

Logs en formato NDJSON convertidos a DataFrame:

	ts	user	event	ok	query	item_id
2024-05-01	10:00:01+00:00	u001	login	True	NaN	NaN
2024-05-01	10:00:45+00:00	u001	search	NaN	laptop	NaN
2024-05-01	10:01:10+00:00	u001	click	NaN	NaN	P-9821
2024-05-01	10:02:00+00:00	u002	login	False	NaN	NaN
2024-05-01	10:03:15+00:00	u001	compra	NaN	NaN	P-9821

Patron de sesion de u001 (secuencia de eventos en orden):
login -> search -> click -> compra

Mismo resultado con pd.read_json(lines=True):

	ts	user	event
2024-05-01T10:00:01Z	u001	login	
2024-05-01T10:00:45Z	u001	search	
2024-05-01T10:01:10Z	u001	click	
2024-05-01T10:02:00Z	u002	login	
2024-05-01T10:03:15Z	u001	compra	

```
# — XML: lectura con ElementTree —————
#
# XML es el formato obligatorio en sectores como sanidad (HL7/FHIR),
# administracion publica (facturacion electronica), finanzas (XBRL)
# y automatizacion industrial (OPC-UA).
```

```
xml_catalogo = """<?xml version="1.0" encoding="UTF-8"?>
```

```
<catalogo>
  <producto id="P001" categoria="electronica">
    <nombre>Laptop Pro X</nombre>
    <precio moneda="EUR">1299.99</precio>
    <stock>42</stock>
    <especificaciones>
      <ram>16GB</ram>
      <almacenamiento>512GB SSD</almacenamiento>
      <cpu>Intel Core i7</cpu>
    </especificaciones>
  </producto>
  <producto id="P002" categoria="accesorios">
    <nombre>Raton Ergonomico</nombre>
    <precio moneda="EUR">45.50</precio>
    <stock>180</stock>
    <especificaciones>
      <conexion>Bluetooth</conexion>
      <dpi>1600</dpi>
    </especificaciones>
  </producto>
  <producto id="P003" categoria="electronica">
    <nombre>Monitor 4K</nombre>
    <precio moneda="EUR">389.00</precio>
    <stock>15</stock>
    <especificaciones>
      <pulgadas>27</pulgadas>
      <resolucion>3840x2160</resolucion>
      <panel>IPS</panel>
    </especificaciones>
```

```

    </producto>
</catalogo>"""

root = ET.fromstring(xml_catalogo)

# Convertir a DataFrame: iterar sobre elementos del XML
registros_xml = []
for prod in root.findall('producto'):
    fila = {
        'id': prod.get('id'),
        'categoria': prod.get('categoria'),
        'nombre': prod.findtext('nombre'),
        'precio': float(prod.findtext('precio')),
        'stock': int(prod.findtext('stock')),
    }
    # Aplanar especificaciones (campos variables segun producto)
    specs = prod.find('especificaciones')
    if specs is not None:
        for spec in specs:
            fila[f'spec_{spec.tag}'] = spec.text
    registros_xml.append(fila)

df_xml = pd.DataFrame(registros_xml)
print("Catalogo XML convertido a DataFrame:")
print(df_xml.to_string(index=False))
print()
print("Columnas con NaN = especificaciones que no existen en todos los productos.")
print("Esto es tipico al aplanar XML/JSON con esquema variable.")
print()
print(f"Productos electronicos: {len(df_xml[df_xml['categoria']=='electronica'])}")
print(f"Valor total en stock: {(df_xml['precio'] * df_xml['stock']).sum():.2f} EUR")

```

Catalogo XML convertido a DataFrame:

id	categoria	nombre	precio	stock	spec_ram	spec_almacenamiento
spec_cpu	spec_conexion	spec_dpi	spec_pulgadas	spec_resolucion	spec_panel	
P001	electronica	Laptop Pro X	1299.99	42	16GB	512GB SSD Intel
Core i7	NaN	NaN	NaN	NaN	NaN	NaN
P002	accesorios	Raton Ergonomico	45.50	180	NaN	
NaN	NaN	Bluetooth	1600	NaN	NaN	NaN
P003	electronica	Monitor 4K	389.00	15	NaN	
NaN	NaN	NaN	NaN	27	3840x2160	IPS

Columnas con NaN = especificaciones que no existen en todos los productos.
 Esto es tipico al aplanar XML/JSON con esquema variable.

Productos electronicos: 2

Valor total en stock: 68624.58 EUR

2.4 Formatos orientados a ML y Big Data: Parquet, Arrow y HDF5

Los formatos clásicos (CSV, JSON, XML) fueron diseñados para intercambio, no para análisis a gran escala. Los formatos modernos priorizan:

- **Almacenamiento columnar:** para acceder solo a las columnas necesarias sin leer toda la fila.
- **Compresión nativa:** reducir el tamaño sin sacrificar velocidad de lectura.

- **Esquema integrado:** los tipos de datos se guardan junto con los datos (no hay ambigüedad).
- **Lectura parcial (pushdown):** filtrar filas antes de descomprimirlas.

Los tres grandes formatos para ML

Formato	Orientación	Soporte en pandas	Caso de uso principal en ML
Parquet	Disco columnar	<code>read_parquet()</code> / <code>to_parquet()</code>	Almacenamiento de datasets de entrenamiento
Apache Arrow	Memoria columnar	<code>pa.Table</code> , <code>pa.RecordBatch</code>	Transferencia zero-copy entre frameworks
HDF5	Arrays numéricos	<code>pd.read_hdf()</code> / <code>h5py</code>	Pesos de modelos, matrices densas, datos científicos

Parquet es el estándar de facto en pipelines ML modernos: S3 en Parquet + pandas/Spark/DuckDB es la combinación más común en data lakes. Delta Lake e Apache Iceberg añaden transacciones ACID sobre Parquet.

```
# — Benchmark real: CSV vs Parquet vs Arrow – tamaño y velocidad —
#
# Generamos un dataset sintético de 500.000 filas y lo guardamos
# en distintos formatos. Medimos tamaño en disco y tiempo de lectura.

try:
    import pyarrow as pa
    import pyarrow.parquet as pq
    HAS_ARROW = True
except ImportError:
    HAS_ARROW = False

rng = np.random.default_rng(0)
N = 500_000
df_bench = pd.DataFrame({
    'id': np.arange(N),
    'categoria': rng.choice(['A', 'B', 'C', 'D', 'E'], N),
    'valor_1': rng.normal(100, 15, N).round(4),
    'valor_2': rng.exponential(50, N).round(4),
    'valor_3': rng.integers(0, 10_000, N),
    'flag': rng.integers(0, 2, N).astype(bool),
    'texto_corto': rng.choice(['alpha', 'beta', 'gamma', 'delta', 'epsilon'], N),
})
print(f"Dataset de benchmark: {N:,} filas x {df_bench.shape[1]} columnas")

rutas = {
    'CSV (sin comprimir)': '/tmp/bench.csv',
    'CSV (gzip)': '/tmp/bench.csv.gz',
    'Parquet (snappy)': '/tmp/bench.parquet',
    'Parquet (gzip)': '/tmp/bench_gz.parquet',
}

if HAS_ARROW:
    rutas['Arrow IPC'] = '/tmp/bench.arrow'

resultados = {}

# Escritura
df_bench.to_csv('/tmp/bench.csv', index=False)
```

```

df_bench.to_csv('/tmp/bench.csv.gz', index=False, compression='gzip')
df_bench.to_parquet('/tmp/bench.parquet', compression='snappy', index=False)
df_bench.to_parquet('/tmp/bench_gz.parquet', compression='gzip', index=False)
if HAS_ARROW:
    table = pa.Table.from_pandas(df_bench)
    with pa.ipc.new_file('/tmp/bench.arrow', table.schema) as writer:
        writer.write_table(table)

# Medicion
for nombre, ruta in rutas.items():
    if not os.path.exists(ruta):
        continue
    tamaño_mb = os.path.getsize(ruta) / 1_048_576

    t0 = time.perf_counter()
    if ruta.endswith('.csv') or ruta.endswith('.csv.gz'):
        _ = pd.read_csv(ruta)
    elif ruta.endswith('.parquet'):
        _ = pd.read_parquet(ruta)
    elif ruta.endswith('.arrow') and HAS_ARROW:
        with pa.ipc.open_file(ruta) as f:
            _ = f.read_pandas()
    t_read = (time.perf_counter() - t0) * 1000

    resultados[nombre] = {'tamaño_mb': tamaño_mb, 't_lectura_ms': t_read}

# Tabla comparativa
print()
print(f"{'Formato':<25} {'Tamano (MB)':>12} {'Lectura (ms)':>13} {'Ratio tamano':>10}")
print("-" * 68)
csv_size = resultados['CSV (sin comprimir)']['tamaño_mb']
for nombre, vals in resultados.items():
    ratio = vals['tamaño_mb'] / csv_size
    print(f"{'nombre':<25} {'vals['tamaño_mb']':>12.1f} {'vals['t_lectura_ms']':>13.0f} {'ratio':>12.2f}x")

```

Dataset de benchmark: 500,000 filas x 7 columnas

Formato	Tamano (MB)	Lectura (ms)	Ratio tamano
CSV (sin comprimir)	19.8	277	1.00x
CSV (gzip)	6.9	385	0.35x
Parquet (snappy)	9.7	187	0.49x
Parquet (gzip)	6.5	84	0.33x
Arrow IPC	22.1	91	1.11x

```

# — Grafico del benchmark —————
if resultados:
    nombres = list(resultados.keys())
    tamanios = [resultados[n]['tamaño_mb'] for n in nombres]
    tiempos = [resultados[n]['t_lectura_ms'] for n in nombres]

    colores_bench = {
        'CSV (sin comprimir)': '#B71C1C',
        'CSV (gzip)': '#E53935',
        'Parquet (snappy)': '#2E7D32',
        'Parquet (gzip)': '#1B5E20',
        'Arrow IPC': '#1565C0',
    }
    bar_colors = [colores_bench.get(n, '#546E7A') for n in nombres]

```

```

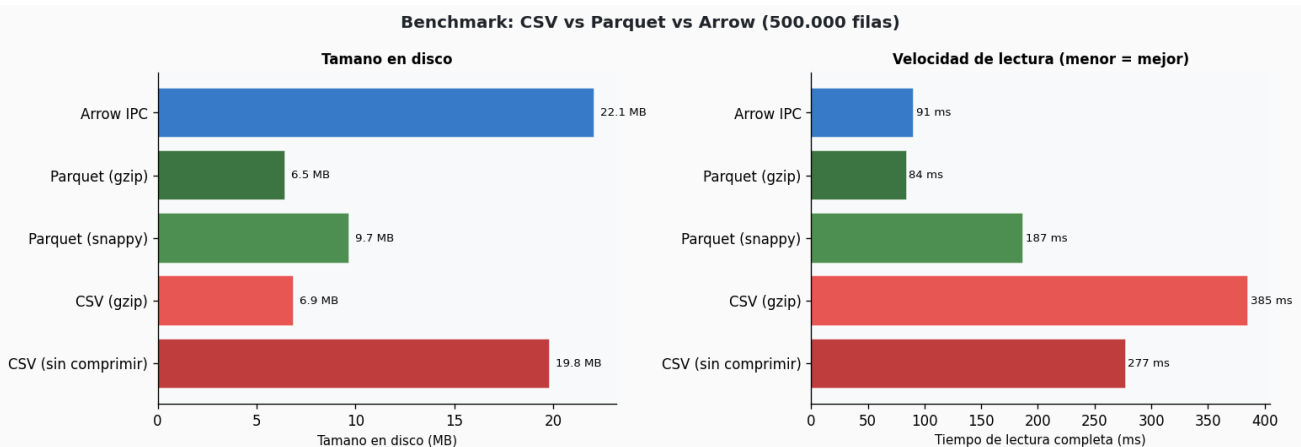
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4.5))
fig.patch.set_facecolor('#FAFAFA')
fig.suptitle('Benchmark: CSV vs Parquet vs Arrow (500.000 filas)',
             fontsize=12, fontweight='bold', color='#212529')

# Tamano en disco
bars1 = ax1.barh(nombres, tamanios, color=bar_colors, alpha=0.85, edgecolor='white')
ax1.set_xlabel('Tamano en disco (MB)', fontsize=9)
ax1.set_title('Tamano en disco', fontsize=10, fontweight='bold')
ax1.set_facecolor('#F8F9FA')
ax1.spines[['top', 'right']].set_visible(False)
for bar, v in zip(bars1, tamanios):
    ax1.text(v + 0.3, bar.get_y() + bar.get_height() / 2,
             f'{v:.1f} MB', va='center', fontsize=8)

# Tiempo de lectura
bars2 = ax2.barh(nombres, tiempos, color=bar_colors, alpha=0.85, edgecolor='white')
ax2.set_xlabel('Tiempo de lectura completa (ms)', fontsize=9)
ax2.set_title('Velocidad de lectura (menor = mejor)', fontsize=10, fontweight='bold')
ax2.set_facecolor('#F8F9FA')
ax2.spines[['top', 'right']].set_visible(False)
for bar, v in zip(bars2, tiempos):
    ax2.text(v + 2, bar.get_y() + bar.get_height() / 2,
             f'{v:.0f} ms', va='center', fontsize=8)

plt.tight_layout()
plt.show()
else:
    print("No hay resultados de benchmark disponibles.")

```



```

# — Apache Arrow: columnar en memoria y transferencia zero-copy —————
try:
    import pyarrow as pa
    import pyarrow.compute as pc

    print("=== Apache Arrow: columnar en memoria ===")
    print()

    # Arrow Table desde pandas (copia minima de datos)
    t0 = time.perf_counter()
    tabla = pa.Table.from_pandas(df_bench)
    t_conv = (time.perf_counter() - t0) * 1000
    print(f"Conversion pandas -> Arrow: {t_conv:.1f} ms")
    print(f"Schema Arrow: {tabla.schema}")
    print()

```

```

# Operacion columnar sin conversion: suma de valor_1 por categoria
t0 = time.perf_counter()
# Filtrar solo categoria 'A' usando Arrow compute
mask      = pc.equal(tabla.column('categoria'), 'A')
tabla_a   = tabla.filter(mask)
suma_a    = pc.sum(tabla_a.column('valor_1')).as_py()
t_arrow   = (time.perf_counter() - t0) * 1000

t0 = time.perf_counter()
suma_pd   = df_bench.loc[df_bench['categoria'] == 'A', 'valor_1'].sum()
t_pandas  = (time.perf_counter() - t0) * 1000

print(f"Suma valor_1 (cat=A):")
print(f"  Arrow: {suma_a:.4f} en {t_arrow:.1f} ms")
print(f"  pandas: {suma_pd:.4f} en {t_pandas:.1f} ms")
print()

# Slicing zero-copy: no copia datos
columna_val1 = tabla.column('valor_1')
slice_val1   = columna_val1.slice(0, 5)
print(f"Primeros 5 valores de valor_1 (zero-copy slice): {slice_val1.to_pylist}")
print()
print("-> Zero-copy: pandas y Arrow pueden compartir el mismo buffer de memoria")
print("  Esto es clave en pipelines donde intervienen Spark, Dask o Ray.")

```

```

except ImportError:
    print("PyArrow no instalado. Para instalarlo: pip install pyarrow")
    print()
    print("Apache Arrow define un formato columnar en memoria estandar.")
    print("Permite transferencia de datos sin copias entre:")
    print("  pandas <-> Spark <-> Dask <-> DuckDB <-> Polars <-> Ray")

```

=== Apache Arrow: columnar en memoria ===

Conversion pandas -> Arrow: 81.9 ms

Schema Arrow: id: int64

categoria: string

valor_1: double

valor_2: double

valor_3: int64

flag: bool

texto_corto: string

-- schema metadata --

pandas: '{"index_columns": [{"kind": "range", "name": null, "start": 0, "' + 1067

Suma valor_1 (cat=A):

Arrow: 9983141.7906 en 10.9 ms

pandas: 9983141.7906 en 30.8 ms

Primeros 5 valores de valor_1 (zero-copy slice): [86.365, 105.5361, 113.4853, 106.9961, 116.7458]

-> Zero-copy: pandas y Arrow pueden compartir el mismo buffer de memoria.
 Esto es clave en pipelines donde intervienen Spark, Dask o Ray.

```

# — HDF5: almacenamiento de arrays numericos grandes con h5py —————
try:
    import h5py

    print("=== HDF5: almacenamiento jerarquico de arrays ===")

```

```

print()

# HDF5 organiza los datos en grupos (directorios) y datasets (arrays).
# Es el formato estandar para pesos de modelos (Keras .h5),
# datos climaticos, astronomia, neurociencia.

ruta_h5 = '/tmp/experimento_ml.h5'

# — Escritura —
with h5py.File(ruta_h5, 'w') as f:
    # Grupo para datos de entrenamiento
    grp_train = f.create_group('datos/entrenamiento')
    rng_h5 = np.random.default_rng(99)
    X_train = rng_h5.normal(0, 1, (10_000, 20)).astype(np.float32)
    y_train = rng_h5.integers(0, 3, 10_000)

    ds_X = grp_train.create_dataset('X', data=X_train,
                                     compression='gzip', compression_opts=4)
    ds_y = grp_train.create_dataset('y', data=y_train)

    # Metadatos como atributos
    ds_X.attrs['descripcion'] = '10.000 muestras x 20 features (float32)'
    ds_X.attrs['normalizacion'] = 'StandardScaler (media=0, std=1)'
    f.attrs['proyecto'] = 'Clasificacion multiclase - prueba UD1'
    f.attrs['fecha'] = '2026-08-14'
    f.attrs['autor'] = 'ETCDFM 2026-27'

    # Grupo para resultados del experimento
    grp_res = f.create_group('resultados')
    grp_res.create_dataset('accuracies', data=np.array([0.78, 0.82, 0.85, 0.87,
                                                         0.89, 0.91, 0.93, 0.95, 0.97, 0.99]))
    grp_res.create_dataset('losses', data=np.array([1.20, 0.95, 0.78, 0.65,
                                                     0.50, 0.40, 0.30, 0.20, 0.10, 0.05]))

tamano_kb = os.path.getsize(ruta_h5) / 1024
print(f"Fichero HDF5 creado: {tamano_kb:.1f} KB")
print()

# — Lectura selectiva (solo una parte) —
with h5py.File(ruta_h5, 'r') as f:
    # Introspection del fichero
    print("Estructura del fichero HDF5:")
    f.visititems(lambda name, obj:
                  print(f"  {name} [{type(obj).__name__}]"))

    print()
    print("Atributos del fichero:")
    for k, v in f.attrs.items():
        print(f"  {k}: {v}")

    # Lectura parcial: solo las primeras 5 filas de X (sin cargar todo)
    X_parcial = f['datos/entrenamiento/X'][:5, :]
    print()
    print(f"Primeras 5 filas de X (lectura parcial, shape: {X_parcial.shape}):")
    print(np.round(X_parcial[:, :6], 3)) # primeras 6 columnas

    accs = f['resultados/accuracies'][:5]
    print()
    print(f"Curva de accuracy: {accs}")
    print(f"Mejor accuracy: {accs.max():.2%}")

except ImportError:

```

```
print("h5py no instalado. Para instalarlo: pip install h5py")
print()
print("HDF5 (Hierarchical Data Format v5) se usa para:")
print(" - Guardar pesos de redes neuronales (Keras: model.save('model.h5'))")
print(" - Datasets científicos grandes (NetCDF, datos climáticos, MRI)")
print(" - Arrays numéricos con metadatos y compresión integrada")
print(" - Estructura jerárquica: grupos = directorios, datasets = arrays")
```

h5py no instalado. Para instalarlo: `pip install h5py`

HDF5 (Hierarchical Data Format v5) se usa para:

- Guardar pesos de redes neuronales (Keras: `model.save('model.h5')`)
- Datasets científicos grandes (NetCDF, datos climáticos, MRI)
- Arrays numéricos con metadatos y compresión integrada
- Estructura jerárquica: grupos = directorios, datasets = arrays

2.5 Sistemas IoT y SCADA: datos industriales para ML

Los sistemas **SCADA** (Supervisory Control and Data Acquisition) y las redes **IoT** son la principal fuente de datos en **ML industrial** (mantenimiento predictivo, detección de anomalías, optimización de procesos).

Arquitectura en capas del mundo industrial

Nivel 4: Empresa / Nube / ML

MES, ERP, Data Lake, modelos de predicción

Nivel 3: Supervisión (SCADA / Historian)

SCADA software, BD de series temporales (InfluxDB, TimescaleDB)

Nivel 2: Control (PLCs / RTUs)

Lógica de control, alarmas, setpoints

Nivel 1: Campo (sensores / actuadores)

Temperatura, presión, vibración, caudal, corriente

Protocolos y formatos de datos

Protocolo	Capa	Datos que genera	Formato habitual
OPC-UA	2-3	Valores de tags con timestamp	XML / binario
MQTT	1-3	Mensajes ligeros (IoT)	JSON / binario
Modbus	1-2	Registros numéricos de PLCs	Binario
IEC 61850	2-3	Subestaciones eléctricas	XML (SCL)

Tipo de datos que generan: **series temporales**

Cada sensor produce una lectura cada N segundos/ms: `{timestamp, sensor_id, valor}`. Las características clave son:

- **Alta frecuencia:** 1 Hz a 10 kHz (vibración de motor).
- **Multivariate:** decenas/cientos de sensores simultáneos.
- **Anomalías raras:** la tasa de fallos es $\ll 1\%$ → dataset muy desbalanceado.

Arquitectura IoT / SCADA para ML industrial

Modelos ML: mantenimiento predictivo | detección de anomalías | optimización de procesos | calidad de producto

XGBoost / LSTM / Autoencoder / Isolation Forest

Nivel 4 — Empresa / Cloud / ML

Data Lake (S3/ADLS) | Apache Spark | MLflow | Feature Store

Datos: datasets para entrenamiento, predicciones en batch

Nivel 3 — Supervision (SCADA / Historian)

SCADA software | InfluxDB | TimescaleDB | OSIsoft PI

Datos: series temporales comprimidas, agregaciones

Nivel 2 — Control (PLCs / RTUs / MQTT)

Logica de control | Alarmas | Setpoints | Modbus / OPC-UA

OPC-UA / MQTT

Datos: tags con timestamp, eventos de alarma

Nivel 1 — Sensores y actuadores

Temperatura | Presion | Vibracion | Caudal | Corriente

OPC-UA / Modbus

Datos: registros numericos en bruto a alta frecuencia

— Simulacion de datos SCADA: serie temporal multivariada de una turbina —

```
import random
from collections import deque

random.seed(42)
np.random.seed(42)

# Parametros de la planta: turbina de vapor en central electrica
SENSORES = {
    'temp_entrada_c': {'media': 450, 'std': 8, 'unidad': 'C'},
    'temp_salida_c': {'media': 120, 'std': 5, 'unidad': 'C'},
    'presion_bar': {'media': 32, 'std': 0.8, 'unidad': 'bar'},
    'vibracion_mms': {'media': 2.5, 'std': 0.4, 'unidad': 'mm/s'},
    'rpm': {'media': 3000, 'std': 15, 'unidad': 'rpm'},
    'potencia_mw': {'media': 50, 'std': 3, 'unidad': 'MW'},
}

# Generamos 30 minutos de datos a 1 Hz = 1800 lecturas
N_LECTURAS = 1800
timestamps = pd.date_range('2026-05-14 08:00:00', periods=N_LECTURAS, freq='s')

filas = []
# Inyectamos un fallo gradual en vibracion entre t=900 y t=1200 (5-20 min)
for i, ts in enumerate(timestamps):
    fila = {'timestamp': ts, 'turbina_id': 'TB-03'}
    for sensor, params in SENSORES.items():
        val = np.random.normal(params['media'], params['std'])
        # Simular degradacion progresiva de vibracion (rodamiento deteriorado)
        if sensor == 'vibracion_mms' and 900 <= i < 1200:
            val += (i - 900) * 0.012 # aumento lineal de 3.6 mm/s en 5 min
        elif sensor == 'vibracion_mms' and i >= 1200:
            val += 3.6 + np.random.normal(0, 0.3) # vibracion elevada estable
        fila[sensor] = round(val, 3)
    filas.append(fila)

df_scada = pd.DataFrame(filas)
df_scada = df_scada.set_index('timestamp')

print("Serie temporal SCADA: turbina TB-03 (primeras 5 lecturas):")
print(df_scada.head().to_string())
print(f"\nShape total: {df_scada.shape} | Duracion: {N_LECTURAS} segundos (30 min)")
print()
```

```
# Estadísticas descriptivas por periodo (normal vs anomalia vs critico)
df_scada['periodo'] = pd.cut(
    np.arange(N_LECTURAS), bins=[0, 900, 1200, 1800],
    labels=['Normal (0-15min)', 'Degradacion (15-20min)', 'Critico (20-30min)']
)
print("Estadísticas de vibracion por periodo:")
resumen = df_scada.groupby('periodo', observed=True)['vibracion_mms'].agg(
    media='mean', std='std', max='max'
).round(3)
print(resumen.to_string())
```

Serie temporal SCADA: turbina TB-03 (primeras 5 lecturas):

	turbina_id	temp_entrada_c	temp_salida_c	presion_bar
vibracion_mms	rpm	potencia_mw		
timestamp				
2026-05-14 08:00:00	TB-03	453.974	119.309	32.518
3.109 2996.488	49.298			
2026-05-14 08:00:01	TB-03	462.634	123.837	31.624
2.717 2993.049	48.603			
2026-05-14 08:00:02	TB-03	451.936	110.434	30.620
2.275 2984.808	50.943			
2026-05-14 08:00:03	TB-03	442.736	112.938	33.173
2.410 3001.013	45.726			
2026-05-14 08:00:04	TB-03	445.645	120.555	31.079
2.650 2990.990	49.125			

Shape total: (1800, 7) | Duracion: 1800 segundos (30 min)

Estadísticas de vibracion por periodo:

	media	std	max
periodo			
Normal (0-15min)	2.486	0.411	4.070
Degradacion (15-20min)	4.276	1.103	6.888
Critico (20-30min)	6.102	0.490	7.676

```
# — Visualizacion de la serie temporal y extraccion de features para ML —
fig, axes = plt.subplots(3, 1, figsize=(13, 7), sharex=True)
fig.patch.set_facecolor('#FAFAFA')
fig.suptitle('Serie temporal SCADA: turbina TB-03 – deteccion de anomalia',
    fontsize=12, fontweight='bold', color='#212529')

t = np.arange(N_LECTURAS) / 60 # tiempo en minutos

# Calcular media movil de vibracion (ventana 60s)
vibracion = df_scada['vibracion_mms'].values
ventana = 60
media_mov = np.convolve(vibracion, np.ones(ventana) / ventana, mode='same')
umbral = SENSORES['vibracion_mms']['media'] + 3 * SENSORES['vibracion_mms']['std']

for ax, sensor, color in [
    (axes[0], 'temp_entrada_c', '#C62828'),
    (axes[1], 'presion_bar', '#1565C0'),
    (axes[2], 'vibracion_mms', '#E65100'),
]:
    vals = df_scada[sensor].values
    ax.plot(t, vals, alpha=0.4, color=color, linewidth=0.6, label='lectura 1Hz')
    if sensor == 'vibracion_mms':
        ax.plot(t, media_mov, color='#6A1B9A', linewidth=1.5,
```

```

        label='media movil 60s')
    ax.axhline(umbral, color='#B71C1C', linewidth=1.5,
               linestyle='--', label=f'umbral alarma ({umbral:.1f})')
    # Sombrear zona anomala
    ax.axvspan(15, 20, alpha=0.12, color='#E65100', label='degradacion')
    ax.axvspan(20, 30, alpha=0.18, color='#B71C1C', label='zona critica')
    ax.set_ylabel(f"{sensor}\n({SENSORES[sensor]['unidad']})", fontsize=8)
    ax.set_facecolor('#F8F9FA')
    ax.spines[['top', 'right']].set_visible(False)
    ax.legend(fontsize=7, loc='upper left')

```

```

axes[2].set_xlabel('Tiempo (minutos)', fontsize=9)
plt.tight_layout()
plt.show()

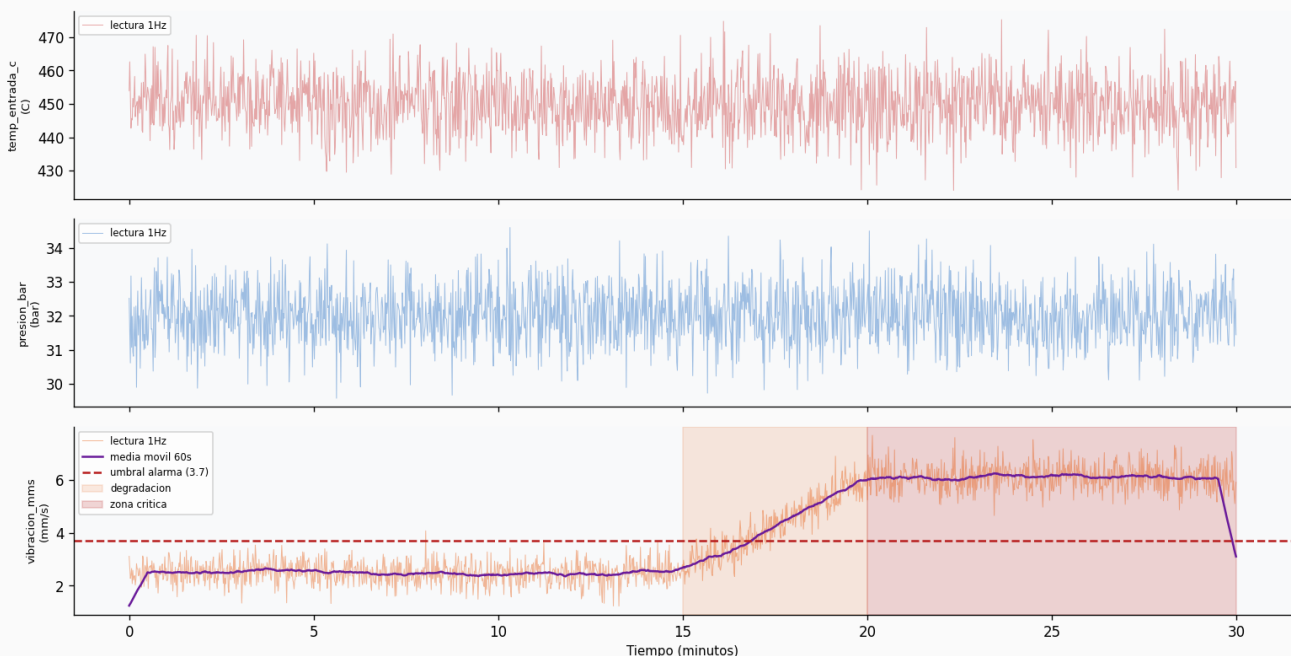
```

```

print()
print("Features tipicas para un modelo de deteccion de anomalias en series temporal")
print()
print("  Estadisticas de ventana (rolling window):")
print("    - media movil, desviacion tipica, min, max en ventanas de 10s, 60s, 5min")
print()
print("  Features de frecuencia (FFT):")
print("    - amplitud de las frecuencias dominantes (vibracion de rodamientos)")
print()
print("  Features cruzadas:")
print("    - diferencia temp_entrada - temp_salida (eficiencia termica)")
print("    - vibracion / rpm (vibracion normalizada por velocidad)")
print()
print("  Modelos habituales:")
print("    - Isolation Forest, Autoencoder (deteccion no supervisada)")
print("    - LSTM, Temporal Fusion Transformer (prediccion de series temporales)")

```

Serie temporal SCADA: turbina TB-03 — deteccion de anomalia



Features típicas para un modelo de detección de anomalías en series temporales:

Estadísticas de ventana (rolling window):

- media móvil, desviación típica, min, max en ventanas de 10s, 60s, 5min

Features de frecuencia (FFT):

- amplitud de las frecuencias dominantes (vibración de rodamientos)

Features cruzadas:

- diferencia temp_entrada - temp_salida (eficiencia térmica)
- vibración / rpm (vibración normalizada por velocidad)

Modelos habituales:

- Isolation Forest, Autoencoder (detección no supervisada)
- LSTM, Temporal Fusion Transformer (predicción de series temporales)

Resumen del Bloque 2

Concepto	Idea clave
BD relacionales (SQL)	Tablas con esquema fijo. <code>pd.read_sql_query()</code> extrae DataFrames. SQLite para local; DuckDB para analítica; PostgreSQL para producción.
BD NoSQL documental	Documentos JSON anidados. Esquema flexible. <code>pd.json_normalize()</code> aplanar para ML.
BD NoSQL clave-valor	Lookup O(1). Patrón de feature store online: escribe en batch, lee en inferencia.
BD NoSQL grafos	Nodos + aristas. NetworkX en Python. Detecta ciclos, calcula PageRank como features.
CSV	Ubicuo pero frágil: encoding, separador, nulos variables. Siempre explorar antes de <code>read_csv()</code> .
JSON	Soporta datos anidados. NDJSON = un objeto por línea, ideal para logs y streaming.
XML	Verboso pero estándar en sectores legacy (HL7, XBRL, OPC-UA). <code>ElementTree</code> + iteración.
Parquet	Columnar + comprimido + esquema integrado. Estándar de facto en data lakes ML. 3-10x menor que CSV.
Apache Arrow	Columnar en memoria. Zero-copy entre pandas, Spark, DuckDB. Formato de transferencia.
HDF5	Arrays numéricos jerárquicos. Estándar para pesos de modelos Keras y datos científicos.
SCADA / IoT	Series temporales multivariadas de alta frecuencia. Mantenimiento predictivo, detección de anomalías.

Siguiente bloque

Bloque 3 — Plataformas de procesamiento masivo y entornos distribuidos: arquitecturas SMP/MPP, introducción a Apache Spark y servicios cloud (S3, BigQuery, Data Factory).